

TUTORIAL 04

Prometheus and Grafana

DDM501 — AI in DevOps, DataOps, MLOps

1. Nothing to see

The API is running and has been up for three weeks. Answer these:

1. How many predictions has it served?
2. What fraction of requests are failing?
3. Is it slower than it was last Tuesday?
4. Which model version is serving right now?
5. Is it still producing roughly the same mix of answers as when you deployed it?

Nothing was written down because nothing was watching. The process is running, and that is the whole of what you know.

This runs the other way: start with no metrics and add them one at a time.

Setup

```
python scripts/train_model.py  # once, before anything else
uvicorn app.main:app --port 8000 # sections 1-4 and 6 need nothing else
docker compose up --build      # section 5 needs Prometheus and Grafana
Ports are 18000 / 19090 / 13000 under Docker, shifted from Lab 4's so both can run.
```

569 rows draw no graphs. Leave the load generator running in its own terminal while you look at anything.

```
python scripts/traffic.py --rps 20 --seconds 300
python scripts/traffic.py --broken 0.2          # 20% malformed requests
python scripts/traffic.py --drift 3.0           # shift every input by 3 sigma
```

2. The /metrics page

```
curl localhost:8000/metrics

# HELP wdbc_predictions_total Predictions served, by outcome.
# TYPE wdbc_predictions_total counter
wdbc_predictions_total{outcome="benign"} 298.0
wdbc_predictions_total{outcome="malignant"} 171.0
wdbc_errors_total{reason="missing_features"} 40.0
wdbc_prediction_latency_seconds_bucket{le="0.0025"} 421.0
wdbc_prediction_latency_seconds_bucket{le="0.005"} 469.0
wdbc_prediction_latency_seconds_count 469.0
wdbc_prediction_latency_seconds_sum 0.9901291770002274
wdbc_model_loaded 1.0
wdbc_model_info{sklearn_version="1.6.0",version="1.0.0"} 1.0
wdbc_malignant_share 0.365
```

That is the entire interface: a text page, fetched on a schedule. Your service sends nothing anywhere and does not know Prometheus exists.

Prometheus pulls. A service that stops answering shows up as an absence, which is why `up == 0` is the first alert anyone writes.

3. Three metric types

Almost everything you need is one of three. The skill is picking the right one.

3.1 Counter

```
PREDICTIONS = Counter("wdbc_predictions_total",  
                        "Predictions served, by outcome.", ["outcome"])  
PREDICTIONS.labels(outcome="malignant").inc()
```

Only goes up, and is almost never read directly. What you read is its rate:

```
sum by (outcome) (rate(wdbc_predictions_total[1m]))
```

The `outcome` label is what makes an error rate expressible at all: two series under one name. Questions 1 and 2.

3.2 Histogram

An average of 40 ms fits everyone getting 40 ms, and it fits 5% getting 600 ms. Only one of those is an incident. A histogram keeps the shape:

```
wdbc_prediction_latency_seconds_bucket{le="0.001"}    0.0  
wdbc_prediction_latency_seconds_bucket{le="0.0025"}  421.0  
wdbc_prediction_latency_seconds_bucket{le="0.005"}   469.0  
wdbc_prediction_latency_seconds_count                469.0
```

421 of 469 requests finished within 2.5 ms and all within 5 ms, so p95 sits between those edges. The average, 2.1 ms, hides the tail.

```
histogram_quantile(0.95,  
  sum(rate(wdbc_prediction_latency_seconds_bucket[5m])) by (le))
```

Buckets are a decision

`histogram_quantile` interpolates between bucket edges, so its accuracy depends on whether your buckets bracket the values you care about. The library defaults run to 10 seconds, which this service never reaches; ours stop at 1s.

3.3 Gauge

```
MODEL_LOADED.set(1)  
MODEL_INFO.labels(version="1.0.0", sklearn_version="1.6.0").set(1)  
MALIGNANT_SHARE.set(sum(recent) / len(recent))
```

Goes up and down. The middle one is the info pattern: a gauge stuck at 1 whose labels carry the payload. It answers "which version was serving at 14:20" with a line on a graph instead of a guess. Question 4.

`wdbc_malignant_share` moves with the data rather than the traffic. Question 5, and the only metric here that exists because this is a machine learning service.

4. Four alerts

Each rule comes from a question someone would be woken up for, not from a metric that happened to exist.

Alert	Fires when	Why it is separate
ApiDown	up == 0 for 1m	The absence of a scrape. Nothing has to report its own death.
ModelNotLoaded	wdbc_model_loaded == 0 for 2m	The process answers, so a port check calls it healthy, while every prediction 503s.
HighErrorRate	error share > 5% for 5m	A ratio, not a count. 40 errors is fine at 10k requests and a crisis at 100.
SlowPredictions	p95 > 50 ms for 10m	The tail, not the average — the users who are actually suffering.
MalignantShareShift	share moves > 15 points for 15m	Nothing is broken. The answers changed. Only ML services have this failure mode.

Every rule has a `for:` clause. Without it a single scrape above the threshold pages you. Nearly every false page in the world is a missing `for:`.

The last alert catches what the others cannot: latency fine, errors fine, container healthy, and the model quietly answering differently than it did in April.

5. Watch it happen

With the stack up and traffic running, do these in order and watch the dashboard.

Run	What moves
<code>--broken 0.2</code>	The error-share panel climbs past 5%. After five minutes HighErrorRate fires.
<code>--drift 3.0</code>	Latency flat. Error rate flat. Malignant share moves a long way. Nothing is broken.
<code>docker compose stop api</code>	Predictions per second falls to zero and ApiDown fires a minute later.

The middle row is the one to sit with. Every engineering signal says healthy, because it is. The inputs changed, and the only thing that noticed was a metric someone thought to add.

6. Label cardinality

The most common expensive mistake in Prometheus. Every distinct combination of metric name and label values is one time series, and time series are what Prometheus stores, indexes and holds in memory.

Count what the service exposes with sensible labels:

```
python scripts/traffic.py --rps 30 --seconds 20 --broken 0.1
python scripts/count_series.py

wdbc_errors_total                1
wdbc_malignant_share             1
wdbc_model_info                  1
wdbc_model_loaded                1
wdbc_prediction_latency_seconds_bucket  11
wdbc_predictions_total           2
...
TOTAL                            23 time series
```

Twenty-three, and it does not grow with traffic. Now add one label that looks perfectly reasonable — which patient — and send the same requests:

```
T04_TRAP=1 uvicorn app.main:app --port 8000
python scripts/traffic.py --rps 30 --seconds 20 --broken 0.1
python scripts/count_series.py

wdbc_predictions_total           310
...
TOTAL                            639 time series
```

	Sensible labels	With sample_id
Time series	23	639
/metrics payload	4,106 bytes	52,481 bytes
Growth	constant	one new series per patient, forever

Twenty-seven times the series for identical traffic. Size is not the point: 23 stays 23 whether you serve a hundred requests a day or a million, while the other grows with every new patient and is never collected. Teams find this out when the monitoring falls over.

The rule

A label may only take values from a small, bounded, known-in-advance set: outcome, status code, model version, region. Never an id, an email, a URL with a parameter in it, a timestamp, or anything a user supplies. The test: can you write down every value this label will ever have?

Set T04_TRAP back to 0 and restart before continuing.